

Indian Institute of Technology Kharagpur

AUTUMN Semester, 2026–2027

COMPUTER SCIENCE AND ENGINEERING

Computer Organization and Architecture Laboratory

MIPS32 Assignment 4

July 2026

Instructions: *This is an individual assignment. Each student must demonstrate the working programs to a Teaching Assistant for evaluation. Submit a single zipped folder named `MIPSAssignment4_{Your_Roll_No}.zip`. The folder must contain the complete and well-commented MIPS32 assembly source files for both questions. Each source file must include a header containing the student's name, roll number, assignment number, question number, and date of submission.*

For both questions, the program must contain a `main` section and a separate procedure. The `main` section must perform input, validation, procedure invocation, result printing, and program termination. The required computation must be performed inside the specified procedure.

Question 1: Prime Number Test Using a Procedure

1. Objective

Write a MIPS32 assembly-language program that determines whether a given integer greater than 1 is a prime number.

The prime-number test must be implemented in a separate procedure named

`is_prime`

and must determine divisibility through repeated subtraction. Multiplication, division, and remainder instructions must not be used.

2. Problem Definition

A prime number is an integer greater than 1 that has exactly two positive divisors: 1 and itself. For example,

2, 3, 5, 7, 11, 13, ...

are prime numbers, whereas

4, 6, 8, 9, 10, ...

are composite numbers.

Possible divisors must be tested beginning from 2. For each divisor, divisibility must be determined through repeated subtraction.

3. Required Procedure Interface

The procedure must use the following interface:

Item	Requirement
Procedure name	<code>is_prime</code>
Input	<code>\$a0</code> contains an integer $N > 1$
Output	<code>\$v0</code> =1 if N is prime <code>\$v0</code> =0 if N is composite
Invocation	The procedure must be called using <code>jal is_prime</code>
Return	The procedure must return using <code>jr \$ra</code>

The procedure must be a leaf procedure and must not invoke another procedure.

4. Algorithm

Algorithm 1 `is_prime` procedure using repeated subtraction

Require: An integer $N > 1$

Ensure: Return 1 if N is prime; otherwise return 0

```
1: if  $N = 2$  then
2:   return 1
3:  $D \leftarrow 2$ 
4: while  $D \neq N$  do
5:    $R \leftarrow N$ 
6:   while  $R > 0$  do
7:      $R \leftarrow R - D$ 
8:   if  $R = 0$  then
9:     return 0
10:   $D \leftarrow D + 1$ 
11: return 1
```

5. Assignment Task

Write a MIPS32 assembly-language program that performs the following operations:

1. Ask the user to enter an integer greater than 1.
2. Read the integer using syscall 5.
3. Verify that the input is greater than 1.
4. Print an error message and terminate if the input is less than or equal to 1.
5. Preserve the original input value in `main`.
6. Place the validated input in `$a0`.
7. Call the procedure using `jal is_prime`.
8. Perform the complete prime-number test inside `is_prime`.
9. Return the result through `$v0`.
10. Print whether the original input is prime or composite.
11. Terminate the program using syscall 10.

6. Suggested Control-Flow Organization

The program should follow this general organization:

```
.data
    # Input prompt
    # Prime-number message
    # Composite-number message
    # Invalid-input message
    # Newline string

.text
.globl main

main:
    # Read the input
    # Validate the input
    # Preserve the original value
    # Place the argument in $a0

    jal is_prime

    # Examine the returned value in $v0
    # Print the appropriate result
    # Terminate using syscall 10

is_prime:
    # Input:  $a0 = integer greater than 1
    # Output: $v0 = 1 for prime, 0 for composite

    # Perform the repeated-subtraction test

    jr $ra

invalid_input:
    # Print the error message
    # Terminate using syscall 10
```

The listing specifies only the required organization. Students must complete the MIPS32 instructions independently.

7. Input Validation

The input must be an integer greater than 1.

The following inputs are invalid:

- zero;
- one; and
- any negative integer.

For invalid input, the program must print a message similar to:

```
Error: The input must be an integer greater than 1.
```

Input validation must be completed in `main` before calling `is_prime`.

8. Allowed Instructions and Directives

The solution may primarily use:

- `.data`, `.text`, `.globl`, and `.ascii`;
- `la`, `li`, and `move`;
- `add`, `addi`, `addu`, `addiu`, and `sub`;
- `beq`, `bne`, `bgt`, `bge`, `blt`, `ble`, and `bltz`;
- `b`;
- `jal`;
- `jr`; and
- `syscall`.

Equivalent pseudo-instructions supported by MARS or QtSPIM may be used.

9. Restrictions

The following instructions and methods must not be used:

- `mul`;
- `mult` or `multu`;
- `div` or `divu`;
- `rem`;
- `mflo` or `mfhi`;
- a precomputed table of prime numbers;
- hard-coded answers for individual inputs;
- conversion of the input to a string for divisibility testing;
- macros that perform the prime-number test;
- recursion;
- calling `is_prime` using an unconditional branch or jump instead of `jal`; and
- performing the prime-number computation directly in `main`.

The `is_prime` procedure must not print messages, read input, or terminate the program. It must only perform the required computation and return its result through `$v0`.

10. Sample Runs

Sample Run 1

Enter an integer greater than 1: 29 29 is a prime number.
--

Sample Run 2

```
Enter an integer greater than 1: 21
21 is a composite number.
```

Sample Run 3

```
Enter an integer greater than 1: 2
2 is a prime number.
```

Sample Run 4: Invalid Input

```
Enter an integer greater than 1: 1
Error: The input must be an integer greater than 1.
```

11. Required Test Cases

Input	Value returned in \$v0	Expected output
2	1	Prime
3	1	Prime
4	0	Composite
7	1	Prime
9	0	Composite
17	1	Prime
21	0	Composite
29	1	Prime
49	0	Composite
1	–	Error message
0	–	Error message
-7	–	Error message

Additional test cases may be used during evaluation.

12. Correctness Requirements

A correct implementation must satisfy the following conditions:

1. The input is read and validated in `main`.
2. Inputs less than or equal to 1 are rejected before the procedure call.
3. The original input value is preserved for result printing.
4. The validated input is passed through `$a0`.
5. The procedure is called using `jal is_prime`.
6. The procedure tests possible divisors beginning from 2.
7. Divisibility is determined only through repeated subtraction.
8. The procedure returns 1 or 0 through `$v0`.
9. The procedure returns using `jr $ra`.

10. The procedure does not invoke another procedure.
11. The result is printed by `main`.
12. The program terminates using syscall 10.

13. Submission Requirement for Question 1

The source file for Question 1 must be named:

`prime_procedure_{Your_Roll_No}.S`

The file must begin with a comment header similar to:

```
# -----  
# Name:  
# Roll Number:  
# Course: Computer Organization and Architecture Laboratory  
# Assignment: MIPS32 Assignment 4  
# Question: 1  
# Problem: Prime-number test using a procedure  
# Date:  
# -----
```

Question 2: Count the Number of Set Bits Using a Procedure

1. Objective

Write a MIPS32 assembly-language program that counts the number of set bits in the binary representation of a non-negative integer.

The set-bit computation must be implemented in a separate procedure named

`count_set_bits`.

A set bit is a bit whose value is 1.

2. Problem Definition

For example, the decimal number 13 has the binary representation

$$13 = (1101)_2.$$

Since the binary representation contains three bits with value 1, the number of set bits is

3.

The procedure must examine the bits using bitwise and shift instructions.

3. Required Procedure Interface

The procedure must use the following interface:

Item	Requirement
Procedure name	<code>count_set_bits</code>
Input	<code>\$a0</code> contains a non-negative integer N
Output	<code>\$v0</code> contains the number of set bits in N
Invocation	The procedure must be called using <code>jal count_set_bits</code>
Return	The procedure must return using <code>jr \$ra</code>

The procedure must be a leaf procedure and must not invoke another procedure.

4. Algorithm

Algorithm 2 `count_set_bits` procedure

Require: A non-negative integer N

Ensure: Return the number of set bits in N

```
1:  $C \leftarrow 0$ 
2:  $X \leftarrow N$ 
3: while  $X \neq 0$  do
4:    $B \leftarrow X \& 1$ 
5:   if  $B \neq 0$  then
6:      $C \leftarrow C + 1$ 
7:    $X \leftarrow X \gg 1$ 
8: return  $C$ 
```

Here, $\gg 1$ denotes a logical right shift by one bit.

5. Assignment Task

Write a MIPS32 assembly-language program that performs the following operations:

1. Ask the user to enter a non-negative integer.
2. Read the input using syscall 5.
3. Reject the input if it is negative.
4. Preserve the original input value in `main`.
5. Place the validated input in `$a0`.
6. Call the procedure using `jal count_set_bits`.
7. Perform the complete set-bit computation inside `count_set_bits`.
8. Return the result through `$v0`.
9. Print the returned result from `main`.
10. Terminate the program using syscall 10.

6. Suggested Control-Flow Organization

The program should follow this general organization:

```
.data
    # Input prompt
    # Output message
    # Invalid-input message
    # Newline string

.text
.globl main

main:
    # Read the input
    # Validate the input
    # Preserve the original value
    # Place the argument in $a0

    jal count_set_bits

    # Obtain the returned result from $v0
    # Print the result
    # Terminate using syscall 10

count_set_bits:
    # Input:  $a0 = non-negative integer
    # Output: $v0 = number of set bits

    # Perform the bit-counting loop

    jr $ra

invalid_input:
    # Print the error message
    # Terminate using syscall 10
```

The listing specifies only the required organization. Students must complete the MIPS32 instructions independently.

7. Input Validation

The input must be a non-negative integer.

Negative integers are invalid and must produce an error message similar to:

Error: The input must be a non-negative integer.

The input zero is valid. The number of set bits in zero is zero.

Input validation must be completed in `main` before calling `count_set_bits`.

8. Allowed Instructions and Directives

The solution may primarily use:

- `.data`, `.text`, `.globl`, and `.ascii`;
- `la`, `li`, and `move`;
- `add`, `addi`, `addu`, and `addiu`;
- `andi`;
- `srl`;
- `beq`, `bne`, `blt`, and `bltz`;
- `b`;
- `jal`;
- `jr`; and
- `syscall`.

Equivalent pseudo-instructions supported by MARS or QtSPIM may be used.

9. Restrictions

The following instructions and methods must not be used:

- any special population-count or bit-count instruction;
- a lookup table containing precomputed bit counts;
- conversion of the integer into a binary string;
- processing the input as a string;
- hard-coded answers for individual inputs;
- macros that perform the set-bit computation;
- recursion;
- calling `count_set_bits` using an unconditional branch or jump instead of `jal`; and
- performing the set-bit computation directly in `main`.

The set-bit count must be obtained by repeatedly testing the least significant bit and logically shifting the working value to the right.

The `count_set_bits` procedure must not print messages, read input, or terminate the program. It must only perform the required computation and return the result through `$v0`.

10. Sample Runs

Sample Run 1

```
Enter a non-negative integer: 13
The number of set bits is: 3
```

Sample Run 2

```
Enter a non-negative integer: 255
The number of set bits is: 8
```

Sample Run 3

```
Enter a non-negative integer: 0
The number of set bits is: 0
```

Sample Run 4: Invalid Input

```
Enter a non-negative integer: -5
Error: The input must be a non-negative integer.
```

11. Required Test Cases

Input	Expected value returned in \$v0
0	0
1	1
2	1
3	2
7	3
8	1
13	3
15	4
16	1
31	5
255	8
1023	10
1024	1
-5	Error message

Additional test cases may be used during evaluation.

12. Correctness Requirements

A correct implementation must satisfy the following conditions:

1. The input is read and validated in `main`.
2. Negative inputs are rejected before the procedure call.
3. Zero is accepted as a valid input.
4. The original input value is preserved for result printing.
5. The validated input is passed through `$a0`.
6. The procedure is called using `jal count_set_bits`.
7. The least significant bit is tested using a bitwise operation.
8. The counter is incremented only when the tested bit is 1.
9. A logical right shift is used to process the next bit.
10. The procedure returns the result through `$v0`.
11. The procedure returns using `jr $ra`.
12. The procedure does not invoke another procedure.
13. The result is printed by `main`.
14. The program terminates using `syscall 10`.

13. Submission Requirement for Question 2

The source file for Question 2 must be named:

`setbits_procedure_{Your_Roll_No}.S`

The file must begin with a comment header similar to:

```
# -----  
# Name:  
# Roll Number:  
# Course: Computer Organization and Architecture Laboratory  
# Assignment: MIPS32 Assignment 4  
# Question: 2  
# Problem: Count set bits using a procedure  
# Date:  
# -----
```

Common Procedure Requirements

For both questions:

1. The program must contain a `main` section and exactly one computational procedure.
2. The argument must be passed through `$a0`.
3. The result must be returned through `$v0`.

4. The procedure must be invoked using `jal`.
5. The procedure must return using `jr $ra`.
6. The computational procedure must be a leaf procedure.
7. The procedure may use `$t0-$t9` as temporary registers.
8. The procedure must not modify `$s0-$s7`.
9. Input, validation, output, and program termination must be performed by `main`.
10. The value returned by the procedure must be used by `main` to produce the final output.

Common Submission Requirements

Submit a single zipped folder named:

`MIPSAssignment4_{Your_Roll_No}.zip`

The folder must contain the following files:

File	Contents
<code>prime_procedure_{Your_Roll_No}.S</code>	Solution for Question 1: Prime Number Test Using a Procedure
<code>setbits_procedure_{Your_Roll_No}.S</code>	Solution for Question 2: Count the Number of Set Bits Using a Procedure